Analyzing E-commerce Consumer Behaviour

Phil (Fei Han) & Doniyorbek Komilov

2026-04-16

```r
# Load libraries
suppressPackageStartupMessages({
library(tidyverse)
library(corrplot)
library(viridis)
library(scales)
library(car)
library(pROC)
library(DHARMa)
library(ResourceSelection)
library(mgcv)
library(arm)
})
```

```
Warning: package 'car' was built under R version 4.5.3

Warning: package 'carData' was built under R version 4.5.3

Warning: package 'pROC' was built under R version 4.5.3

Warning: package 'DHARMa' was built under R version 4.5.3

Warning: package 'ResourceSelection' was built under R version 4.5.3

Warning: package 'mgcv' was built under R version 4.5.3

Warning: package 'arm' was built under R version 4.5.3
```

```r
# --- Clean and Preprocess Data ---
df_clean <- read.csv("online_shoppers_intention.csv") %>%
  mutate(
    # Convert Revenue to Factor
    Revenue = factor(Revenue, levels = c(FALSE, TRUE), labels = c("No Purchase", "Purchase"))

    # Convert Month (Define Order: Feb is the Reference Level)
    Month = factor(Month, levels = c("Feb", "Mar", "May", "June", "Jul", "Aug", "Sep", "Oct"

    # Convert VisitorType and Weekend
    VisitorType = as.factor(VisitorType),
    Weekend = factor(Weekend, levels = c(FALSE, TRUE), labels = c("Weekday", "Weekend")),

    # Create Numeric Label for calculations
    is_purchase = as.numeric(Revenue == "Purchase")
  ) %>%
  drop_na()

# Define a consistent theme for the project
project_theme <- theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    axis.title = element_text(size = 12),
    legend.position = "bottom"
  )
```

```r
# Check for missing values
cat("Missing values in dataset:", sum(is.na(df_clean)), "\n")
```

```
Missing values in dataset: 0
```

```r
# View the distribution of our target variable: Revenue
table(df_clean$Revenue)
```

```
No Purchase    Purchase
      10422        1908
```

```r
df_clean %>%
  count(Revenue) %>%
```

```r
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = Revenue, y = pct, fill = Revenue)) +
  geom_col(width = 0.5, alpha = 0.8) +
  geom_text(aes(label = paste0(round(pct * 100, 1), "%")),
            vjust = -0.5, size = 5) +
  scale_y_continuous(labels = percent, limits = c(0, 1)) +
  scale_fill_manual(values = c("#FF6666", "#00CC66")) +
  labs(title = "Class Imbalance: Purchase vs. No Purchase",
       subtitle = "Dataset is heavily skewed toward non-purchase sessions",
       x = "Transaction Outcome", y = "Proportion") +
  project_theme
```

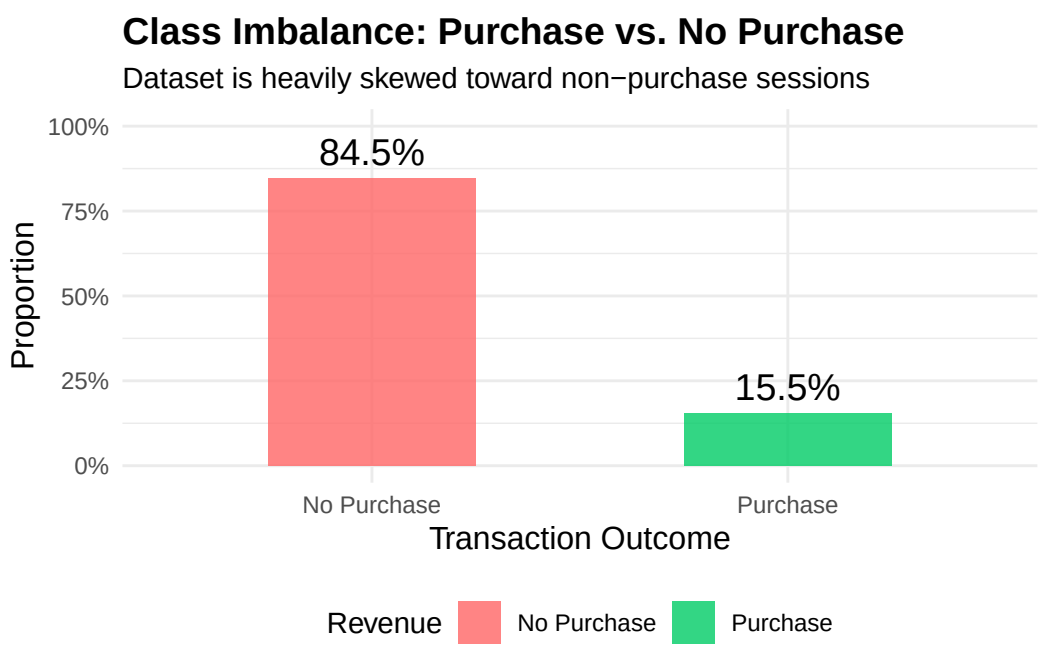


Figure 1: Class Imbalance in Target Variable

```r
ggplot(df_clean, aes(x = Revenue, y = PageValues, fill = Revenue)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = c("#FF6666", "#00CC66")) +
  labs(
    title = "Does Page Value Predict Purchase?",
    subtitle = "Higher PageValues are strongly associated with successful transactions",
    x = "Transaction Outcome (Revenue)",
    y = "Page Value Score"
```

```
  ) +
  project_theme
```

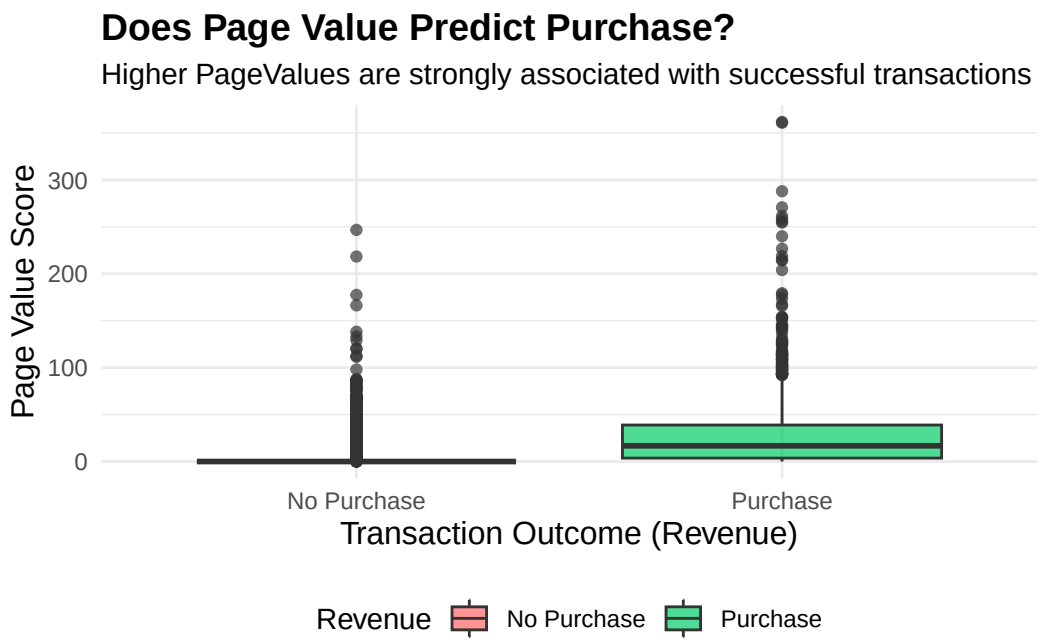


Figure 2: Comparison of Page Values for Purchase vs. No Purchase

```
df_clean %>%
  group_by(Month) %>%
  summarise(Conversion_Rate = mean(Revenue == "Purchase")) %>%
  ggplot(aes(x = Month, y = Conversion_Rate, group = 1)) +
  geom_line(color = "steelblue", size = 1) +
  geom_point(size = 3, color = "darkblue") +
  scale_y_continuous(labels = percent) +
  labs(
    title = "E-commerce Seasonality: Monthly Conversion Rates",
    x = "Month of the Year",
    y = "Conversion Rate (%)"
  ) +
  project_theme
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```

Figure 3: Monthly Conversion Rates Throughout the Year

```
ggplot(df_clean, aes(x = VisitorType, fill = Revenue)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = percent) +
  labs(
    title = "Visitor Behaviour: New vs. Returning",
    x = "Visitor Category",
    y = "Percentage",
    fill = "Purchased?"
  ) +
  project_theme
```

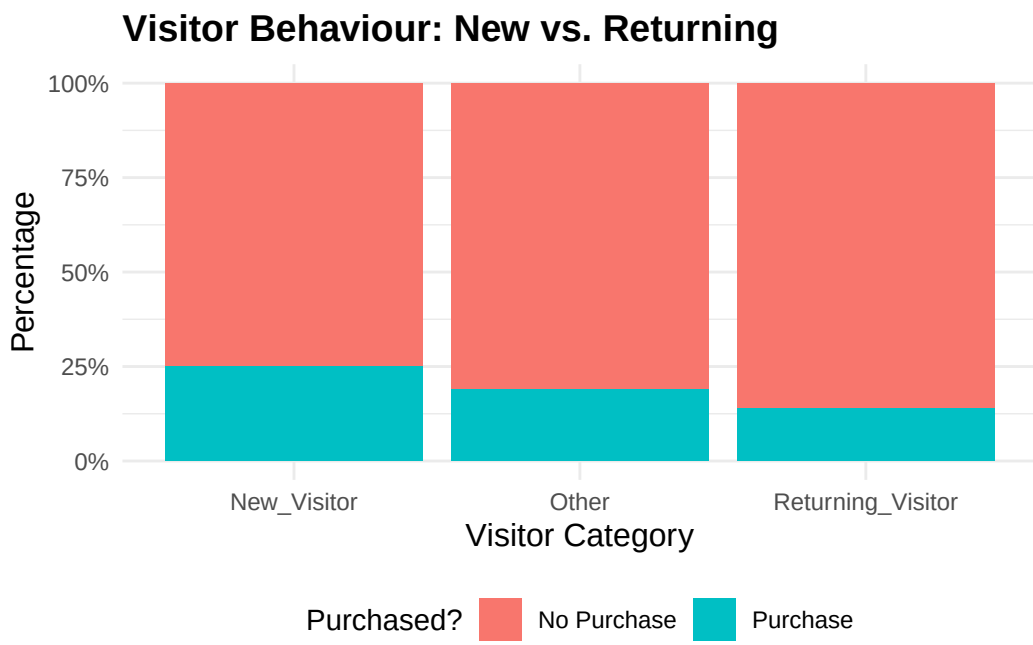


Figure 4: Purchase Proportion by Visitor Type

```
# Select numeric columns and calculate correlation
numeric_df <- df_clean %>% select_if(is.numeric)
cor_matrix <- cor(numeric_df)

corrplot::corrplot(cor_matrix, method = "circle", type = "upper",
                   tl.col = "black", tl.srt = 45,
                   title = "Correlation Heatmap of Website Metrics",
                   mar = c(0,0,1,0))
```

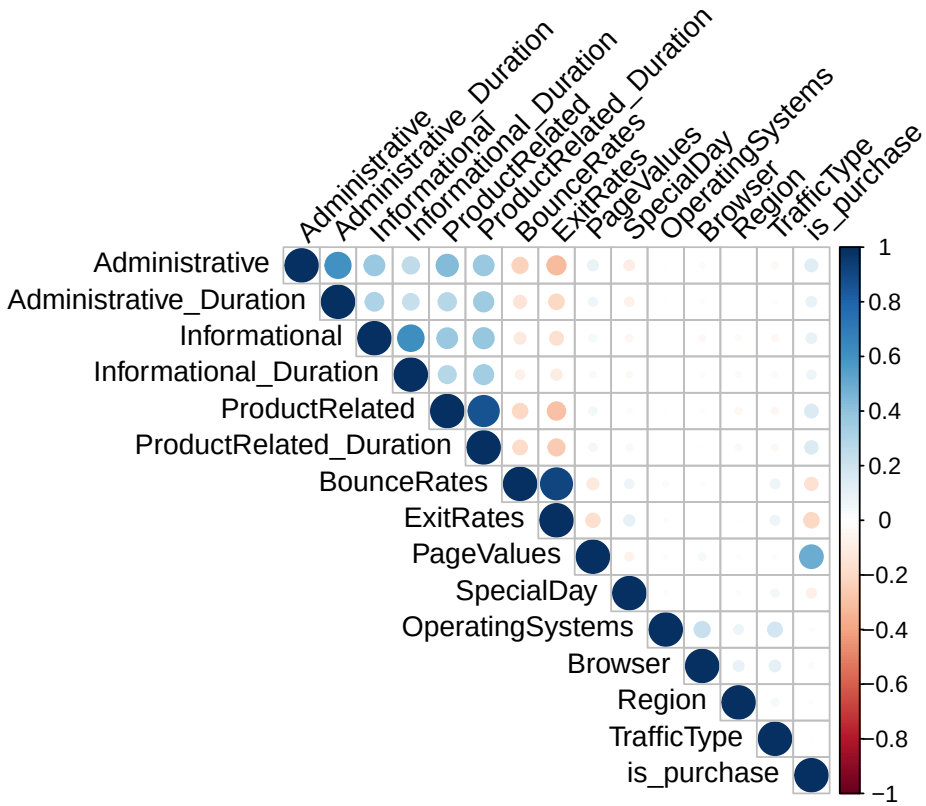


Figure 5

```
df <- read.csv("online_shoppers_intention.csv")

df_clean <- df %>%
  mutate(
    Revenue = factor(Revenue,
                     levels = c(FALSE, TRUE),
```

```r
                      labels = c("No Purchase", "Purchase")),

    Month = factor(Month,
                   levels = c("Feb", "Mar", "May", "June", "Jul", "Aug", "Sep", "Oct", "Nov"

    VisitorType = as.factor(VisitorType),
    Weekend = factor(Weekend,
                      levels = c(FALSE, TRUE),
                      labels = c("Weekday", "Weekend")),

    is_purchase = as.numeric(Revenue == "Purchase")
  ) %>%
  drop_na()
```

```r
summary(df_clean)
```

```
 Administrative   Administrative_Duration Informational
 Min.   : 0.000   Min.   :   0.00         Min.   : 0.0000
 1st Qu.: 0.000   1st Qu.:   0.00         1st Qu.: 0.0000
 Median : 1.000   Median :   7.50         Median : 0.0000
 Mean   : 2.315   Mean   :  80.82         Mean   : 0.5036
 3rd Qu.: 4.000   3rd Qu.:  93.26         3rd Qu.: 0.0000
 Max.   :27.000   Max.   :3398.75         Max.   :24.0000

 Informational_Duration ProductRelated   ProductRelated_Duration
 Min.   :   0.00        Min.   :  0.00   Min.   :    0.0
 1st Qu.:   0.00        1st Qu.:  7.00   1st Qu.:  184.1
 Median :   0.00        Median : 18.00   Median :  598.9
 Mean   :  34.47        Mean   : 31.73   Mean   : 1194.7
 3rd Qu.:   0.00        3rd Qu.: 38.00   3rd Qu.: 1464.2
 Max.   :2549.38        Max.   :705.00   Max.   :63973.5

  BounceRates         ExitRates         PageValues        SpecialDay
 Min.   :0.000000   Min.   :0.00000   Min.   :  0.000   Min.   :0.00000
 1st Qu.:0.000000   1st Qu.:0.01429   1st Qu.:  0.000   1st Qu.:0.00000
 Median :0.003112   Median :0.02516   Median :  0.000   Median :0.00000
 Mean   :0.022191   Mean   :0.04307   Mean   :  5.889   Mean   :0.06143
 3rd Qu.:0.016813   3rd Qu.:0.05000   3rd Qu.:  0.000   3rd Qu.:0.00000
 Max.   :0.200000   Max.   :0.20000   Max.   :361.764   Max.   :1.00000

     Month       OperatingSystems    Browser           Region
```

```
 May    :3364   Min.   :1.000    Min.   : 1.000   Min.   :1.000
 Nov    :2998   1st Qu.:2.000    1st Qu.: 2.000   1st Qu.:1.000
 Mar    :1907   Median :2.000    Median : 2.000   Median :3.000
 Dec    :1727   Mean   :2.124    Mean   : 2.357   Mean   :3.147
 Oct    : 549   3rd Qu.:3.000    3rd Qu.: 2.000   3rd Qu.:4.000
 Sep    : 448   Max.   :8.000    Max.   :13.000   Max.   :9.000
 (Other):1337
  TrafficType                VisitorType       Weekend            Revenue
 Min.   : 1.00   New_Visitor      : 1694   Weekday:9462   No Purchase:10422
 1st Qu.: 2.00   Other            :   85   Weekend:2868   Purchase   : 1908
 Median : 2.00   Returning_Visitor:10551
 Mean   : 4.07
 3rd Qu.: 4.00
 Max.   :20.00

  is_purchase
 Min.   :0.0000
 1st Qu.:0.0000
 Median :0.0000
 Mean   :0.1547
 3rd Qu.:0.0000
 Max.   :1.0000
```

```r
set.seed(42)

n <- nrow(df_clean)
train_idx <- sample(1:n, size = 0.8 * n)

train_data <- df_clean[train_idx, ]
test_data  <- df_clean[-train_idx, ]
train_data$log_PageValues <- log1p(train_data$PageValues)
train_data$log_ExitRates  <- log1p(train_data$ExitRates)
train_data$log_ProductRelated_Duration <- log1p(train_data$ProductRelated_Duration)

test_data$log_PageValues <- log1p(test_data$PageValues)
test_data$log_ExitRates  <- log1p(test_data$ExitRates)
test_data$log_ProductRelated_Duration <- log1p(test_data$ProductRelated_Duration)

df_clean$log_PageValues <- log1p(df_clean$PageValues)
df_clean$log_ExitRates  <- log1p(df_clean$ExitRates)
df_clean$log_ProductRelated_Duration <- log1p(df_clean$ProductRelated_Duration)
```

```r
cat("Training set size:", nrow(train_data), "\n")
```

```
Training set size: 9864
```

```r
cat("Test set size:", nrow(test_data), "\n")
```

```
Test set size: 2466
```

```r
cat("Training set purchase rate:", round(mean(train_data$is_purchase), 3), "\n")
```

```
Training set purchase rate: 0.151
```

```r
cat("Purchase rate of test set:", round(mean(test_data$is_purchase), 3), "\n")
```

```
Purchase rate of test set: 0.171
```

```r
train_data <- read.csv("online_shoppers_intention.csv")


minority_count <- sum(train_data$Revenue == TRUE)
majority_count <- sum(train_data$Revenue == FALSE)
train_data$class_weight <- ifelse(train_data$Revenue == TRUE,
                                  majority_count/minority_count,
                                  1)

final_formula <- Revenue ~ Administrative + ProductRelated + BounceRates + ExitRates + Month
glm_balanced <- glm(final_formula,
                    data    = train_data,
                    family  = quasibinomial,
                    weights = class_weight,
                    control = glm.control(maxit = 200))


library(pROC)
pred_prob <- predict(glm_balanced, type = "response")
roc_curve <- roc(train_data$Revenue, pred_prob)
```

```
Setting levels: control = FALSE, case = TRUE
```

```
Setting direction: controls < cases
```

```r
auc(roc_curve)
```

```
Area under the curve: 0.7418
```

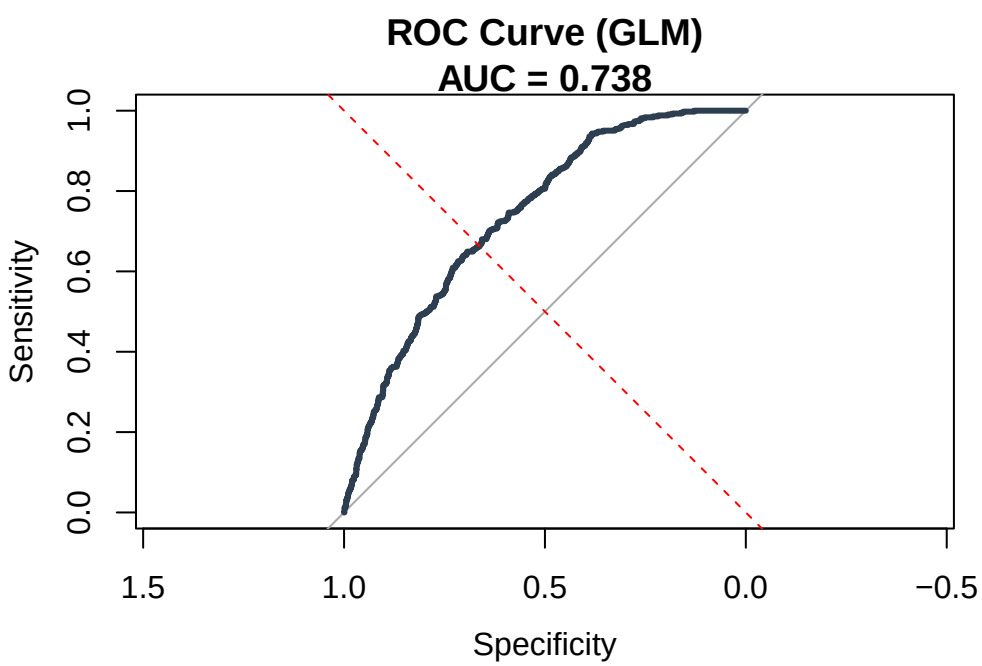
```r
prob_glm <- predict(glm_balanced, newdata = test_data, type = "response")
roc_glm  <- roc(test_data$is_purchase, prob_glm)
```

```
Setting levels: control = 0, case = 1

Setting direction: controls < cases
```

```r
cat("Holdout GLM AUC:", round(auc(roc_glm), 4), "\n")
```

```
Holdout GLM AUC: 0.7378
```

```r
plot(roc_glm, col = "#2c3e50", lwd = 3,
     main = paste("ROC Curve (GLM)\nAUC =", round(auc(roc_glm), 3)))
abline(a = 0, b = 1, lty = 2, col = "red")
```



```r
vif_values <- vif(glm_balanced)

print(vif_values)
```

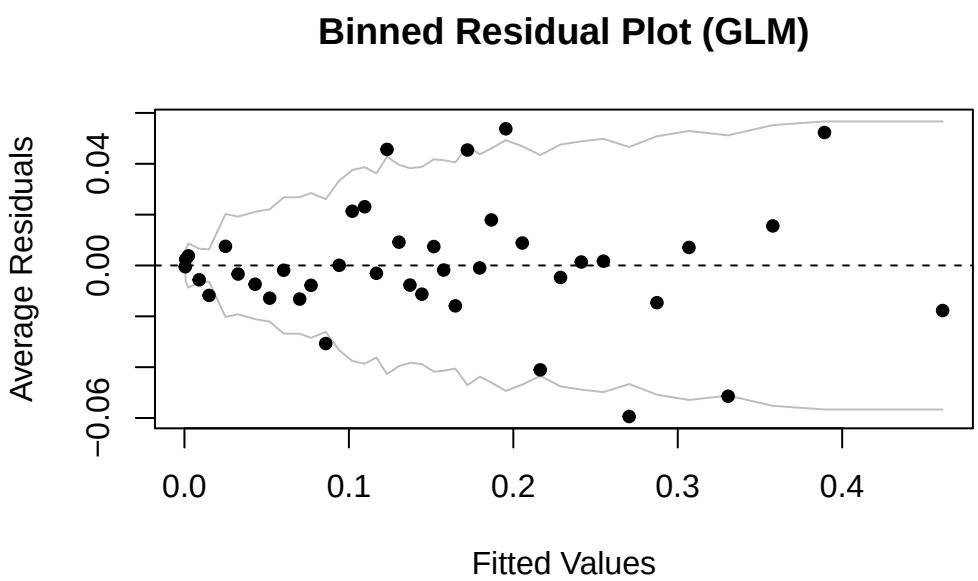
```
                   GVIF Df GVIF^(1/(2*Df))
Administrative 1.197984  1        1.094524
ProductRelated 1.344837  1        1.159671
BounceRates    1.989101  1        1.410355
ExitRates      2.148078  1        1.465632
Month          1.142465  9        1.007427
VisitorType    1.156499  2        1.037018
```

```r
exp_coefficients <- exp(coef(glm_balanced))

print(exp_coefficients)
```

```
                 (Intercept)               Administrative
               2.989450e+00                 1.029243e+00
             ProductRelated                  BounceRates
               1.001960e+00                 2.361454e+01
                ExitRates                     MonthDec
               1.670738e-14                 6.775752e-01
                 MonthFeb                     MonthJul
               1.353126e-01                 1.057463e+00
                MonthJune                     MonthMar
               7.325601e-01                 5.731830e-01
                 MonthMay                     MonthNov
               6.831001e-01                 1.643666e+00
                 MonthOct                     MonthSep
               1.211757e+00                 1.111709e+00
           VisitorTypeOther VisitorTypeReturning_Visitor
               1.284837e+00                 6.716093e-01
```

```r
diag_model <- glm(final_formula,
                  data   = train_data,
                  family = binomial,
                  control = glm.control(maxit = 100))

binnedplot(fitted(diag_model),
           residuals(diag_model, type = "response"),
```

```
            nclass = 40,
            main = "Binned Residual Plot (GLM)",
            xlab = "Fitted Values",
            ylab = "Average Residuals",
            col.int = "grey")
```

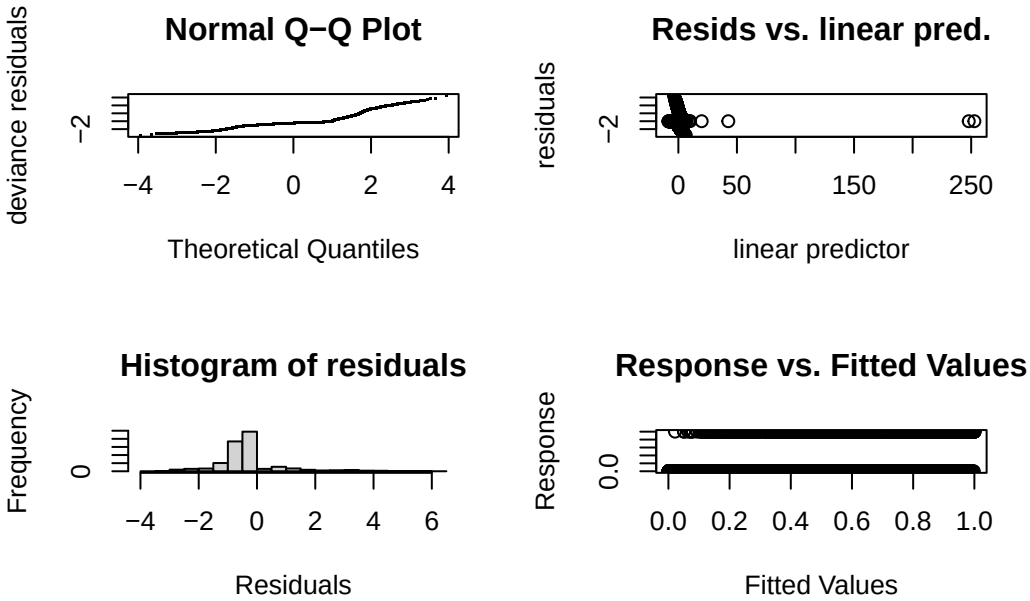



```
all_cols <- colnames(train_data)
cat("train_data ", paste(all_cols, collapse=", "), "\n")
```

```
train_data  Administrative, Administrative_Duration, Informational, Informational_Duration, P
```

```
n_purchase <- sum(train_data$Revenue == TRUE)
n_no_purchase <- sum(train_data$Revenue == FALSE)
train_data$case_weight <- ifelse(
  train_data$Revenue == TRUE,
  n_no_purchase / n_purchase,
  1
)

gam_model <- gam(
  Revenue ~ s(PageValues) + s(ExitRates) + s(ProductRelated_Duration) +
```

```r
    Month + OperatingSystems + VisitorType + Weekend,
  family = quasibinomial,
  data = train_data,
  weights = case_weight
)

gam.check(gam_model)
```




```
Method: GCV   Optimizer: outer newton
full convergence after 8 iterations.
Gradient range [-2.497745e-09,1.082056e-06]
(score 1.255283 & scale 1.316307).
Hessian positive definite, eigenvalue range [2.158171e-05,9.597596e-05].
Model rank =  41 / 41

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

                              k'  edf k-index p-value
s(PageValues)              9.00 8.87    0.87    0.04 *
s(ExitRates)               9.00 8.83    0.88    0.20
```

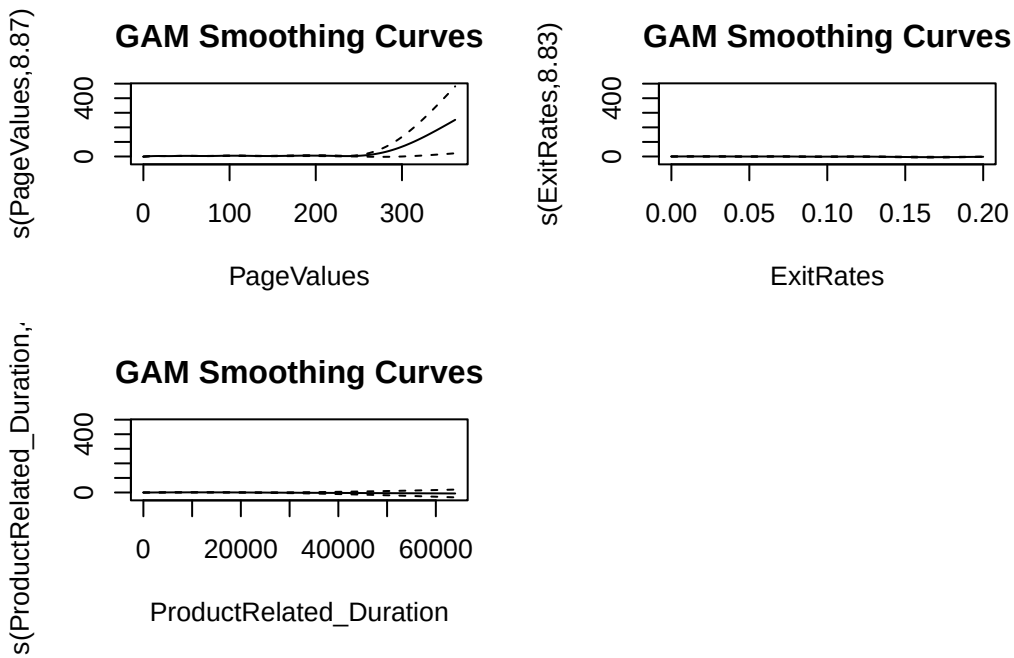
```
s(ProductRelated_Duration) 9.00 4.59    0.91    0.92
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
summary(gam_model)
```

```
Family: quasibinomial
Link function: logit

Formula:
Revenue ~ s(PageValues) + s(ExitRates) + s(ProductRelated_Duration) +
    Month + OperatingSystems + VisitorType + Weekend

Parametric coefficients:
                             Estimate Std. Error t value Pr(>|t|)
(Intercept)                   0.05763    0.14417   0.400   0.6893
MonthDec                     -0.67830    0.13555  -5.004 5.69e-07 ***
MonthFeb                     -2.25134    0.42276  -5.325 1.03e-07 ***
MonthJul                      0.16531    0.16104   1.027   0.3047
MonthJune                    -0.33401    0.20131  -1.659   0.0971 .
MonthMar                     -1.01356    0.13603  -7.451 9.90e-14 ***
MonthMay                     -1.33093    0.12994 -10.243  < 2e-16 ***
MonthNov                      0.66142    0.12308   5.374 7.84e-08 ***
MonthOct                     -0.27529    0.15589  -1.766   0.0774 .
MonthSep                      0.13876    0.15845   0.876   0.3812
OperatingSystems             -0.04973    0.02781  -1.788   0.0737 .
VisitorTypeOther             -0.27125    0.34142  -0.794   0.4269
VisitorTypeReturning_Visitor -0.64756    0.06885  -9.405  < 2e-16 ***
WeekendTRUE                   0.08626    0.05382   1.603   0.1090
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
                             edf Ref.df      F p-value
s(PageValues)              8.871  8.984 405.92  <2e-16 ***
s(ExitRates)               8.830  8.987  17.32  <2e-16 ***
s(ProductRelated_Duration) 4.592  5.584  17.83  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.506   Deviance explained = 46.8%
```

```
GCV = 1.2553  Scale est. = 1.3163    n = 12330
```

```r
plot(gam_model, pages = 1, shade = TRUE, seWithMean = TRUE, main = "GAM Smoothing Curves")
```




```r
set.seed(42)
k      <- 5
folds  <- sample(rep(1:k, length.out = nrow(df_clean)))
auc_glm_cv <- numeric(k)
auc_gam_cv <- numeric(k)

for (i in 1:k) {
  cv_train <- df_clean[folds != i, ]
  cv_test  <- df_clean[folds == i, ]

  np <- sum(cv_train$Revenue == TRUE)
  nn <- sum(cv_train$Revenue == FALSE)
  cv_train$case_weight <- ifelse(cv_train$Revenue == TRUE, nn / np, 1)

  cv_test$case_weight <- 1

  cv_glm <- glm(final_formula,
                data     = cv_train,
```

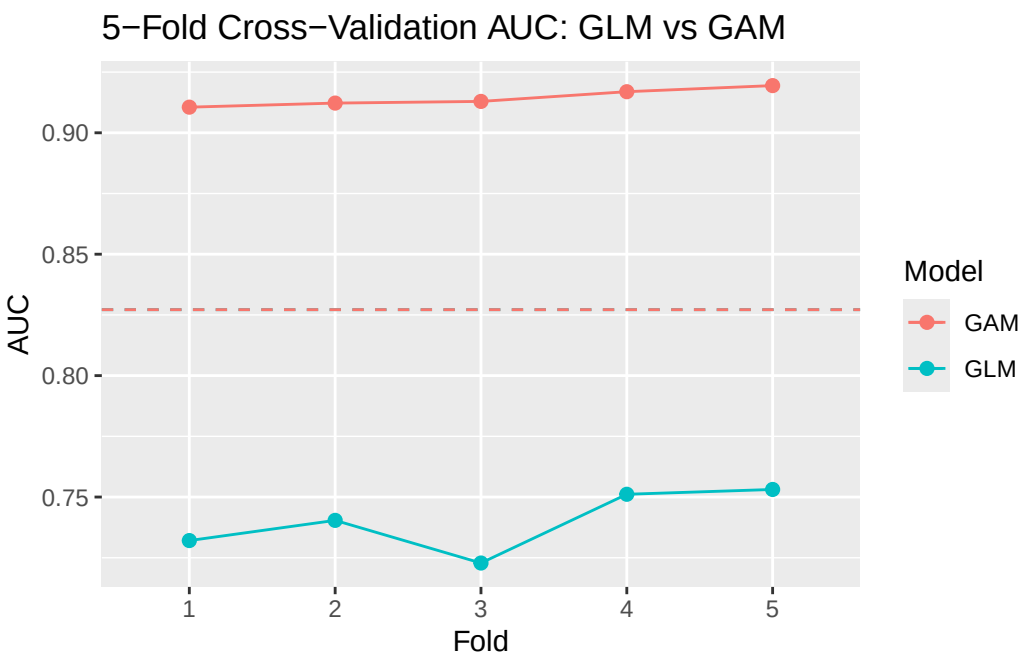
```r
                family  = quasibinomial,
                weights = case_weight,
                control = glm.control(maxit = 200))

  cv_gam <- gam(Revenue ~ s(PageValues) + s(ExitRates) + s(ProductRelated_Duration) +
                  Month + OperatingSystems + VisitorType + Weekend,
                family  = quasibinomial,
                data    = cv_train,
                weights = case_weight)

  p_glm <- predict(cv_glm, newdata = cv_test, type = "response")
  p_gam <- predict(cv_gam, newdata = cv_test, type = "response")

  auc_glm_cv[i] <- as.numeric(auc(roc(cv_test$Revenue, p_glm, quiet = TRUE)))
  auc_gam_cv[i] <- as.numeric(auc(roc(cv_test$Revenue, p_gam, quiet = TRUE)))
}

cat("GLM CV AUC: mean =", round(mean(auc_glm_cv), 4),
    " sd =", round(sd(auc_glm_cv), 4), "\n")
```

```
GLM CV AUC: mean = 0.7399  sd = 0.0128
```

```r
cat("GAM CV AUC: mean =", round(mean(auc_gam_cv), 4),
    " sd =", round(sd(auc_gam_cv), 4), "\n")
```

```
GAM CV AUC: mean = 0.9144  sd = 0.0037
```

```r
cv_results <- data.frame(
  Fold  = rep(1:k, 2),
  AUC   = c(auc_glm_cv, auc_gam_cv),
  Model = rep(c("GLM", "GAM"), each = k)
)

ggplot(cv_results, aes(x = factor(Fold), y = AUC, color = Model, group = Model)) +
  geom_line() +
  geom_point(size = 2) +
  geom_hline(aes(yintercept = mean(AUC), color = Model), linetype = "dashed") +
  labs(title = "5-Fold Cross-Validation AUC: GLM vs GAM",
       x = "Fold", y = "AUC")
```




```
model_comparison <- data.frame(
  Model       = c("GLM (quasibinomial)", "GAM (quasibinomial)"),
  CV_AUC_mean = c(round(mean(auc_glm_cv), 4), round(mean(auc_gam_cv), 4)),
  CV_AUC_sd   = c(round(sd(auc_glm_cv),  4), round(sd(auc_gam_cv),  4))
)
print(model_comparison)
```

```
                Model CV_AUC_mean CV_AUC_sd
1 GLM (quasibinomial)      0.7399    0.0128
2 GAM (quasibinomial)      0.9144    0.0037
```

```
library(pROC)
library(ggplot2)

gam_vars <- all.vars(terms(gam_model)) %>% setdiff("Revenue")
missing_vars <- setdiff(gam_vars, colnames(test_data))
if (length(missing_vars) > 0) {
  stop("The test set is missing required variables
 ", paste(missing_vars, collapse = ", "))
} else {
  cat(" he trial set variables are complete, and the analysis can continue
```

```
n")
}
```

```
 he trial set variables are complete, and the analysis can continue
n
```

```
test_data$Weekend <- as.logical(test_data$Weekend)
train_data$Weekend <- as.logical(train_data$Weekend)
```

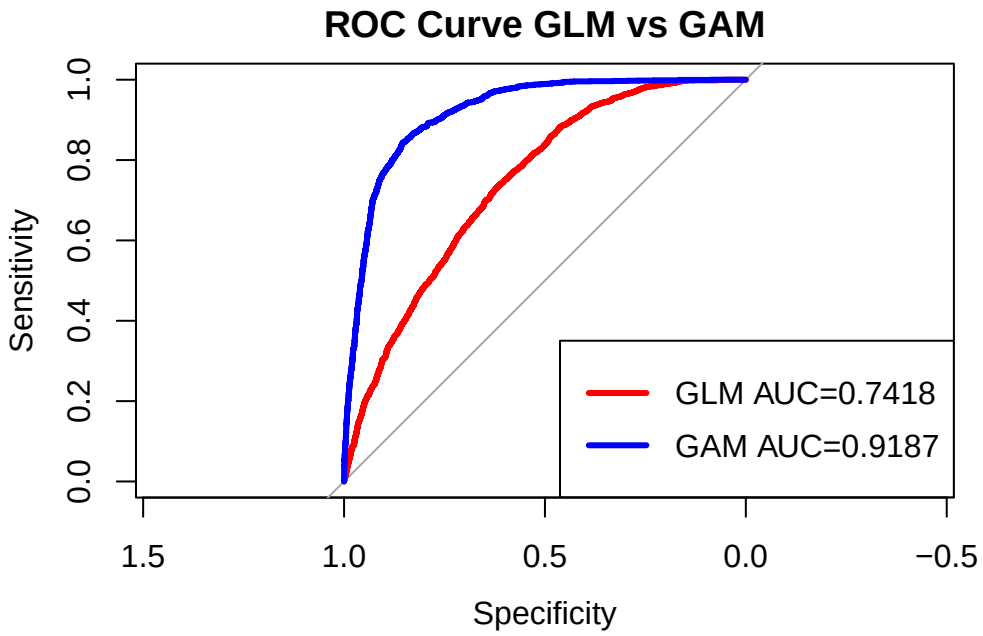
```
library(pROC)

test_data$Weekend <- as.logical(test_data$Weekend)

prob_glm <- predict(glm_balanced, newdata = test_data, type = "response")
prob_gam <- predict(gam_model, newdata = test_data, type = "response")

prob_glm_train <- predict(glm_balanced, newdata = train_data, type = "response")
prob_gam_train <- predict(gam_model, newdata = train_data, type = "response")

roc_glm <- roc(train_data$Revenue, prob_glm_train, quiet=T)
roc_gam <- roc(train_data$Revenue, prob_gam_train, quiet=T)

cat("GLM AUC:", round(auc(roc_glm),4), "\n")
```

```
GLM AUC: 0.7418
```

```
cat("GAM AUC:", round(auc(roc_gam),4), "\n")
```

```
GAM AUC: 0.9187
```

```
plot(roc_glm, col="red", lwd=3, main="ROC Curve GLM vs GAM")
plot(roc_gam, col="blue", lwd=3, add=T)
legend("bottomright",
       c(paste0("GLM AUC=",round(auc(roc_glm),4)),
         paste0("GAM AUC=",round(auc(roc_gam),4))),
       col=c("red","blue"), lwd=3)
```


ROC Curve GLM vs GAM
Sensitivity
0.0
0.2
0.4
0.6
0.8
1.0
1.5
1.0
0.5
0.0
−0.5
Specificity
GLM AUC=0.7418
GAM AUC=0.9187